UNIVERSIDAD CATÓLICA DE CÓRDOBA
JESUITAS

Unidad Académica: FACULTAD DE INGENIERÍA
Asignatura: BASE DE DATOS APLICADA
Año académico: 2026 **Régimen:** SEMESTRAL
Horas Totales: 60 **Carácter:** TEÓRICA/PRÁCTICA

GUÍA DE TRABAJO PRÁCTICO INTEGRADOR

Materia: Base de Datos Aplicada

Temas: Implementación, Administración, Optimización y Programación Procedural en MySQL

1. OBJETIVOS DEL TRABAJO PRÁCTICO

- **Objetivo General:** Desarrollar la implementación completa, operativa, segura y optimizada de un motor de bases de datos relacional (MySQL) a partir de un modelo de dominio previamente diseñado, aplicando técnicas avanzadas de administración, tuning y programación procedural.
- **Objetivos Específicos:**
    - Implementar el diseño físico mediante DDL (SQL) asegurando la integridad referencial y de dominio.
    - Poblar el modelo con un volumen de datos sintéticos representativo utilizando herramientas de Inteligencia Artificial.
    - Configurar esquemas de seguridad basados en roles y privilegios de acceso (DCL).
    - Evaluar y optimizar el rendimiento de consultas utilizando planes de ejecución (EXPLAIN) e índices.
    - Aplicar el concepto de ACID y control de concurrencia mediante transacciones SQL puras.
    - Desarrollar lógica de negocio del lado del servidor mediante Stored Procedures, Functions, Triggers, Manejo de Excepciones y Cursores.

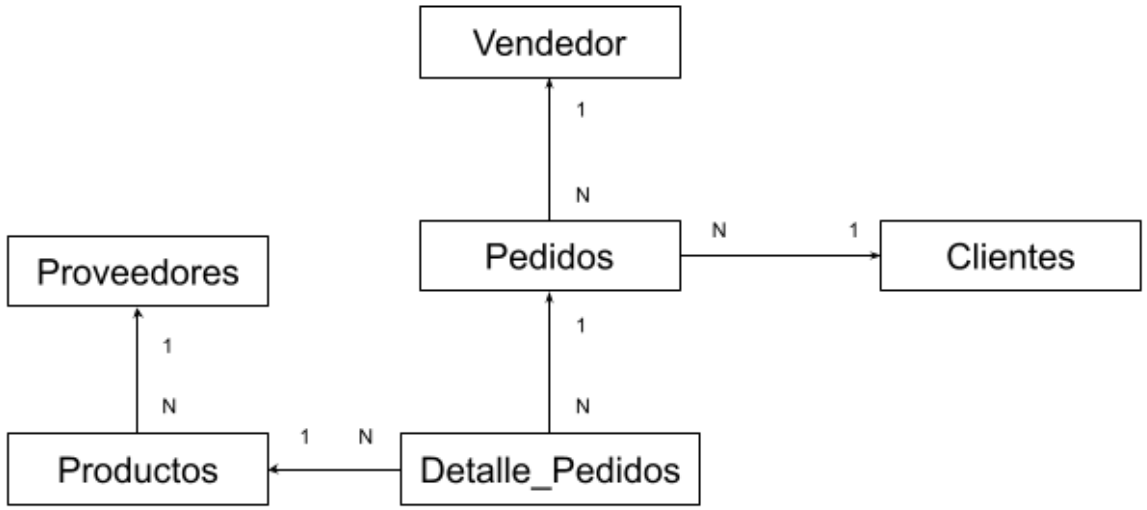
2. ESCENARIO DE NEGOCIO Y MODELO DE DATOS

Una empresa de venta de productos de tecnología se encuentra en la etapa de implementación de su nuevo Sistema de Información. Se requiere poner en producción el módulo de **Registro y Gestión de Pedidos**.

UNIVERSIDAD CATÓLICA DE CÓRDOBA
JESUITAS

Unidad Académica: FACULTAD DE INGENIERÍA
Asignatura: BASE DE DATOS APLICADA
Año académico: 2026 **Régimen:** SEMESTRAL
Horas Totales: 60 **Carácter:** TEÓRICA/PRÁCTICA

Esquema de la base de datos PEDIDOS



Modelo Entidad-Relación (E-R)

- **Clientes** (idcliente, apellido, nombres, direccion, mail)
- **Proveedores** (idproveedor, nombre_proveedor, direccion, mail)
- **Vendedor** (idvendedor, apellido, nombres, mail, comision)
- **Productos** (idproducto, descripcion, precio_unitario, stock, stock_max, stock_min, idproveedor, origen)
- **Pedidos** (numero_pedido, idcliente, idvendedor, fecha, estado)
- **Detalle_Pedidos** (número pedido, renglón, idproducto, cantidad, precio_unitario, total)

Reglas de Negocio Clave

1. Cada proveedor provee artículos de origen 'Nacional' o 'Importado'.
2. Solo se pueden crear pedidos para clientes registrados.
3. Solo se incorporan al detalle productos con stock disponible suficiente.
4. Al confirmar un pedido se debe descontar el stock del producto.
5. El precio unitario en Detalle_Pedidos debe congelar el precio actual del producto al momento de la venta.

6. total en Detalle_Pedidos es un campo calculado (cantidad * precio_unitario).
7. Solo se pueden eliminar clientes que nunca hayan realizado un pedido.
8. Los datos de nuevos productos no admiten campos nulos y el stock inicial debe respetar stock_min < stock < stock_max.
9. El sistema permite la anulación de un pedido. Al anular, se debe recomponer el stock de los productos y auditar la operación registrando fecha y hora en una tabla de log.

PARTE 1: IMPLEMENTACIÓN, SEGURIDAD, OPTIMIZACIÓN Y TRANSACCIONES

(Estimado: Clases 1 y 2)

CLASE 1: DDL, Generación de Datos con IA, Vistas y DCL (Seguridad)

Consigna 1.1: Creación del Esquema Físico (DDL)

1. Crear la base de datos llamada PEDIDOS definiendo el conjunto de caracteres utf8mb4.
2. Escribir el script SQL para la creación de todas las tablas indicadas en el modelo.
3. Definir explícitamente:
    - Claves primarias (PRIMARY KEY) con auto-incremento donde corresponda.
    - Claves foráneas (FOREIGN KEY) con reglas de restricción adecuadas (ON DELETE RESTRICT, ON UPDATE CASCADE).
    - Restricciones de Dominio: CHECK para validar que el origen de productos sea únicamente 'Nacional' o 'Importado', que los precios y stocks sean mayores o iguales a cero, y que la comisión del vendedor esté entre 0 y 100.
    - Cláusulas NOT NULL e índices UNIQUE para correos electrónicos.

Consigna 1.2: Poblado masivo mediante Inteligencia Artificial (DML)

1. Utilizando un asistente de Inteligencia Artificial (ChatGPT, Claude, Gemini, etc.), redacte un prompt para generar un script SQL de inserción de datos sintéticos con las siguientes especificaciones:
    - **Proveedores:** 10 registros.
    - **Vendedores:** 10 registros.
    - **Clientes:** 50 registros.
    - **Productos:** 100 productos con precios entre $5.000 y $1.500.000, stocks coherentes con min y max, y distribución entre 'Nacional' e 'Importado'.

- **Pedidos:** Al menos 200 pedidos distribuidos en el último año, con estados ('CONFIRMADO', 'ANULADO', 'PENDIENTE').
- **Detalle_Pedidos:** Entre 1 y 4 renglones por pedido, garantizando que el precio_unitario y el total concuerden matemáticamente.

2. Ejecute el script generado y verifique la coherencia e integridad de los datos insertados.

Consigna 1.3: Definición de Vistas de Negocio

Crear las siguientes vistas en MySQL:

1. vw_clientes_activos: Listado de clientes que realizaron pedidos entre un rango de fechas dado (incluir Apellido, Nombres, ID, Mail).
2. vw_rendimiento_vendedores: Detalle de vendedores ordenados de mayor a menor según la cantidad total de pedidos gestionados.
3. vw_pedidos_alto_valor: Listado de pedidos cuyo importe total global supere los $500.000.
4. vw_productos_vendidos_periodo: Productos vendidos dentro de un rango de fechas especificando la suma total de unidades vendidas y agrupados por origen.
5. vw_clientes_sin_pedidos: Clientes registrados en el sistema que jamás han realizado un pedido (útil para campañas de fidelización).
6. vw_clientes_ocacionales: Clientes que hayan realizado menos de 2 pedidos en total.

Consigna 1.4: Seguridad y Control de Acceso (DCL)

Configure los esquemas de seguridad del SGBD según las mejores prácticas de administración:

1. Crear el rol rol_auditor con permisos de solo lectura (SELECT) sobre todas las tablas de la base de datos PEDIDOS.
2. Crear el rol rol_vendedor con permisos de consulta y actualización sobre Clientes, Pedidos y Detalle_Pedidos, e inserción sobre Pedidos y Detalle_Pedidos. No debe poder eliminar registros.
3. Crear el rol rol_admin con privilegios totales (ALL PRIVILEGES) sobre la base de datos PEDIDOS.
4. Crear tres usuarios del sistema (usuario_auditoria, usuario_ventas1, usuario_admin1), asignarles sus respectivos roles y verificar los accesos

conectándose desde el cliente terminal o MySQL Workbench probando operaciones permitidas y denegadas.

CLASE 2: Optimización de Consultas (Tuning) y Transacciones SQL

Consigna 1.5: Análisis de Planes de Ejecución e Indexación

1. Ejecute la siguiente consulta sin índices adicionales creados:
2. SQL:

```
SELECT p.descripcion, SUM(dp.cantidad) AS total_vendido
FROM Productos p
JOIN Detalle_Pedidos dp ON p.idproducto = dp.idproducto
JOIN Pedidos pe ON dp.numero_pedido = pe.numero_pedido
WHERE pe.fecha BETWEEN '2024-01-01' AND '2024-06-30'
  AND p.origen = 'Importado'
GROUP BY p.descripcion
ORDER BY total_vendido DESC;
```

3. Anteponga la palabra clave EXPLAIN (o EXPLAIN ANALYZE en MySQL 8.0+) a la consulta anterior.
4. Analice el resultado obtenido:
    - Identifique los tipos de acceso (type: ALL, ref, eq_ref, range, index).
    - Verifique el número de filas examinadas (rows).
    - Observe si existen operaciones costosas en la columna Extra (Using temporary, Using filesort).
5. **Propuesta de Optimización:**
    - Plantee e implemente los índices secundarios (simples o compuestos) necesarios sobre las tablas Pedidos y Productos para optimizar el filtro por fecha y origen.
    - Vuelva a ejecutar EXPLAIN ANALYZE y compare métricas (costo de la consulta, tiempo de ejecución y filas analizadas). Documente los resultados.

UNIVERSIDAD CATÓLICA DE CÓRDOBA
JESUITAS

Unidad Académica: FACULTAD DE INGENIERÍA
Asignatura: BASE DE DATOS APLICADA
Año académico: 2026 **Régimen:** SEMESTRAL
Horas Totales: 60 **Carácter:** TEÓRICA/PRÁCTICA

Consigna 1.6: Práctica de Transacciones ACID mediante Bloques SQL Directos

Sin utilizar procedimientos almacenados, ejecute los siguientes scripts de prueba para analizar el comportamiento del motor de almacenamiento InnoDB ante transacciones explícitas (START TRANSACTION, COMMIT, ROLLBACK y SAVEPOINT).

- **Escenario A (Caso Exitoso - Commit):**
    1. Inicie una transacción explícita.
    2. Inserte un nuevo pedido para el cliente 1 gestionado por el vendedor 1.
    3. Obtenga el numero_pedido generado.
    4. Inserte dos renglones en Detalle_Pedidos.
    5. Descuente las cantidades correspondientes del stock en la tabla Productos.
    6. Aplique un COMMIT. Verifique que las modificaciones persistieron en la base de datos.
- **Escenario B (Caso con Fallo e Integridad - Rollback):**
    1. Inicie una transacción explícita.
    2. Intente insertar un pedido asociándolo a un idcliente inexistente (ej. ID 99999).
    3. Observe el error emitido por la restricción de Clave Foránea (Foreign Key Constraint Failure).
    4. Invoque la sentencia ROLLBACK.
    5. Verifique que ningún cambio parcial afectó el estado de la base de datos.
- **Escenario C (Puntos de Salvaguarda - Savepoint):**
    1. Inicie una transacción.
    2. Actualice el precio de un producto.
    3. Cree un punto de salvaguarda: SAVEPOINT punto1;.
    4. Modifique el stock de otro producto.
    5. Ejecute la sentencia ROLLBACK TO SAVEPOINT punto1;.
    6. Finalice con COMMIT. Compruebe qué cambios se aplican y cuáles se descartaron.

PARTE 2: PROGRAMACIÓN PROCEDURAL (SQL PROCEDURAL)

(Estimado: Clases 3 y 4)

CLASE 3: Stored Procedures, Functions y Manejo de Errores

Consigna 2.1: Funciones Almacenadas (Stored Functions)

1. **fn_calcular_total_pedido(p_numero_pedido INT)**
    - **Descripción:** Función que recibe como parámetro el número de un pedido y retorna el importe total acumulado sumando el campo total de todos sus renglones en Detalle_Pedidos.
    - Debe retornar un valor de tipo DECIMAL(12,2).
    - Si el pedido no posee renglones o no existe, debe retornar 0.00.
2. **fn_validar_stock_disponible(p_idproducto INT, p_cantidad INT)**
    - **Descripción:** Función booleana (retorna TINYINT(1)) que verifica si el stock actual de un producto es mayor o igual a la cantidad solicitada. Retorna 1 (Verdadero) o 0 (Falso).

Consigna 2.2: Procedimientos Almacenados (Stored Procedures) con Gestión de Excepciones y Transacciones

1. **sp_registrar_pedido_completo**
    - **Parámetros de entrada:** p_idcliente INT, p_idvendedor INT, p_idproducto INT, p_cantidad INT.
    - **Requerimientos:**
        - Debe iniciar una transacción explícita.
        - Debe verificar la existencia del cliente y del vendedor.
        - Debe validar la disponibilidad de stock utilizando la función fn_validar_stock_disponible.
        - Si no hay stock suficiente, debe levantar una excepción personalizada utilizando SIGNAL SQLSTATE '45000' indicando el mensaje: *"Error: Stock insuficiente para procesar la operación"*.
        - En caso de error inesperado de SQL (ej. violación de FK, error de tipo de datos), debe declarar un DECLARE HANDLER FOR SQLEXCEPTION

que ejecute automáticamente un ROLLBACK y retorne un mensaje de error claro al usuario.

- Si todas las validaciones son exitosas:
  - Inserta el encabezado en Pedidos con estado 'CONFIRMADO'.
  - Obtiene el ID insertado (LAST_INSERT_ID()).
  - Obtiene el precio unitario actual del producto.
  - Inserta el renglón 1 en Detalle_Pedidos.
  - Descuenta el stock en la tabla Productos.
  - Confirma la transacción mediante COMMIT.

2. **sp_actualizar_precios_por_origen**
   - **Parámetros de entrada:** p_origen VARCHAR(20), p_porcentaje_incremento DECIMAL(5,2).
   - **Requerimientos:**
     - Modifica el precio_unitario de todos los productos pertenecientes al origen especificado ('Nacional' o 'Importado') aplicando el porcentaje indicado (ej. si recibe 15.00, incrementa los precios un 15%).
     - Debe validar que el porcentaje de incremento sea superior a cero. En caso contrario, emitir una excepción con SIGNAL.

CLASE 4: Triggers, Cursores y Auditoría de Sistema

Consigna 2.3: Tabla de Auditoría y Triggers (Disparadores)

1. **Creación de la Tabla de Log:**
   Crear la tabla Log_Anulaciones para auditoría del sistema:
2. SQL:

```
CREATE TABLE Log_Anulaciones (
  idlog INT AUTO_INCREMENT PRIMARY KEY,
  numero_pedido INT NOT NULL,
  fecha_anulacion DATETIME NOT NULL,
  usuario VARCHAR(100) NOT NULL,
  motivo VARCHAR(255) DEFAULT 'Anulación requerida por el cliente'
);
```

3. **Trigger trg_audit_anulacion_pedido:**
    - **Momento:** AFTER UPDATE sobre la tabla Pedidos.
    - **Comportamiento:** Se dispara cuando el campo estado de un pedido cambia de cualquier valor a 'ANULADO'.
    - **Acción:** Inserta un registro en Log_Anulaciones capturando el numero_pedido, la fecha/hora actual (NOW()) y el usuario de la base de datos que ejecutó la acción (CURRENT_USER()).
4. **Procedimiento Almacenado sp_anular_pedido:**
    - **Parámetro:** p_numero_pedido INT.
    - **Requerimientos:**
        - Inicia una transacción.
        - Declara un gestor de excepciones SQLEXCEPTION con ROLLBACK.
        - Verifica que el pedido exista y que su estado actual no sea 'ANULADO'.
        - Actualiza el estado del pedido a 'ANULADO' (esto activará automáticamente el trigger trg_audit_anulacion_pedido).
        - Recompone (devuelve) el stock de cada uno de los productos asociados a los renglones de dicho pedido en Detalle_Pedidos.
        - Ejecuta COMMIT.

Consigna 2.4: Uso de Cursores explícitos en Procedimientos Almacenados

1. **sp_procesar_reajuste_stock_critico**
    - **Descripción:** Implementar un procedimiento almacenado que recorra individualmente todos los productos que se encuentren en estado de **Stock Crítico** (stock < stock_min).
    - **Mecanismo:**
        - Declarar un **Cursor** para seleccionar los campos idproducto, descripcion, stock, stock_min, stock_max de la tabla Productos donde stock < stock_min.
        - Declarar un manejador de fin de cursor (DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin_cursor = 1).
        - Abrir el cursor (OPEN), iterar fila por fila mediante FETCH.
        - Por cada producto detectado en nivel crítico, calcular la cantidad requerida para reponer el inventario al nivel máximo (cantidad_reponer = stock_max - stock).

- Imprimir en la consola (o insertar en una tabla temporal de órdenes de compra Ordenes_Compra_Sugeridas) el detalle de la reposición necesaria por producto.
- Cerrar el cursor (CLOSE) y liberar memoria.

3. CRITERIOS DE EVALUACIÓN Y ENTREGA

- **Modalidad:** Resolución individual o en parejas desde el cliente MySQL Terminal o Workbench.
- **Entregables:**
  1. Script SQL unificado conteniendo las sentencias DDL, DML, DCL, Creación de Vistas y Estructuras Procedurales (SP, Functions, Triggers, Cursores).
  2. Informe técnico breve que contenga la captura y explicación gráfica de los planes de ejecución (EXPLAIN ANALYZE) antes y después de aplicar los índices en la Clase 2.
  3. Código fuente del prompt utilizado para la generación de datos masivos mediante IA.